EXIM Marketplace — Java Backend Explained

A ground-up walkthrough of the API Gateway and one microservice (identity-service), folder by folder, file by file, with every key annotation and method.

Spring Boot 4 · Spring Cloud Gateway · Spring Data MongoDB · JWT security

---

Big Picture First

```
client (browser / app)
      │  every request hits ONE port: 8080
      ▼
┌──────────────┐
│   GATEWAY    │  :8080 — just a router. No DB, no business logic.
│  (Spring     │         Looks at the URL path → forwards to the right service.
│   Cloud GW)  │
└──────┬───────┘
       │ forwards by path, keeping the Authorization header
       ▼
┌──────────────────────────────────────────────────────────────┐
│ identity:8081  catalog:8083  orders:8087 ... (14 services)   │
│ each = its own Spring Boot app + its own MongoDB database    │
└──────────────────────────────────────────────────────────────┘
```

Each microservice is a **mini full-stack app on the server side**, built in layers:

```
HTTP request
   │
   ▼
Controller   ← receives JSON, returns JSON   (the "web" layer)
   │
   ▼
Service      ← business rules, transactions  (the "brain")
   │
   ▼
Repository   ← talks to MongoDB              (the "data" layer)
   │
   ▼
Domain       ← the documents stored in Mongo
```

DTOs carry data in/out, and shared libraries (`common` , `security`) provide things every service reuses (JWT, error handling, response envelope).

Part 1 — The Gateway

Folder: `gateway/`

File	Role
`GatewayApplication.java`	Boots the app
`application.yml`	The routing table (the real logic lives here, not in Java)
`ActivityLoggingFilter.java`	Audit-trail filter — records every API call

GatewayApplication.java

```
@SpringBootApplication          // ← marks this as the Spring Boot entry point
public class GatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(GatewayApplication.class, args);  // starts embedded server
    }
}
```

- **@SpringBootApplication** = the master annotation. It bundles three things: `@Configuration` (this class can define beans), `@EnableAutoConfiguration` (Spring auto-wires Tomcat, JSON, etc.), and `@ComponentScan` (find all `@Component` / `@Service` / `@Controller` classes).
- `main()` just calls `SpringApplication.run(...)`, which starts the embedded web server. **Every** service has this exact shape.

application.yml — the routing table

The gateway is "thin": it has almost no Java. Routing is pure config. Each route is **path → destination URL**:

```
- id: identity
  uri: ${IDENTITY_URI:http://localhost:8081}    # where to forward
  predicates:
    - Path=/api/v1/auth/**,/api/v1/users/**,/api/v1/companies/**
```

Meaning: *any request whose path starts with `/api/v1/auth/...` → forward to identity-service on 8081.*

> **Critical gotcha:** routes are matched **top to bottom, first match wins**. That's why the `verification` route for `/companies/*/verifications` is listed **before** identity's `/companies/**` — otherwise identity would grab it first.

`${IDENTITY_URI:http://localhost:8081}` means "use env var `IDENTITY_URI`, or default to localhost:8081." This is how the same code runs locally and in production.

ActivityLoggingFilter.java

A **filter** — code that runs around every request. Because *all* traffic funnels through the gateway, one filter here logs every user's action without touching the 14 services.

```
@Component                                          // Spring manages this object
public class ActivityLoggingFilter extends OncePerRequestFilter {
    protected void doFilterInternal(req, res, chain) {
        try {
            chain.doFilter(request, response);    // let the request proceed normally
        } finally {
            record(request, response.getStatus()); // AFTER, fire off an audit log
        }
    }
}
```

- **OncePerRequestFilter** guarantees it runs exactly once per request.
- It's **fire-and-forget** (`sendAsync` , errors swallowed) so audit logging can never slow down or break a real request.

Part 2 — A Microservice (identity-service)

Folder structure maps 1:1 to the layers:

```
services/identity-service/
├── pom.xml                          ← this module's dependencies
├── src/main/resources/
│   ├── application.yml              ← config (port, DB, secrets)
│   ├── application-dev.yml          ← dev overrides
│   └── application-prod.yml         ← prod overrides
└── src/main/java/com/eximplatform/
    ├── identity/
    │   ├── IdentityServiceApplication.java   ← entry point
    │   ├── controller/   ← web layer  (AuthController, UserController, ...)
    │   ├── service/      ← business logic (AuthService, UserService, ...)
    │   ├── repository/   ← DB access  (UserRepository, CompanyRepository)
    │   ├── domain/       ← Mongo documents (User, Company, Role enum)
    │   └── dto/          ← request/response shapes
    ├── config/           ← IdentityMongoConfig, seeders
    └── security/         ← identity-only security beans
```

Let's follow a single request — **register a new company** — straight down through the layers.

Layer 0 — the entry point

```
@SpringBootApplication(scanBasePackages = "com.eximplatform")
public class IdentityServiceApplication { main() {...} }
```

Note `scanBasePackages = "com.eximplatform"`. Normally Spring only scans the app's own package. But the shared `common` and `security` libraries live under `com.eximplatform.common` / `com.eximplatform.security`. Scanning the whole root package means those shared beans (JWT filter, error handler) get picked up. **Every service does this** — it's a key convention.

Layer 1 — Controller (AuthController.java)

The web layer. Maps URLs to Java methods.

```
@RestController                      // = @Controller + responses auto-serialized to JSON
@RequestMapping("/api/v1/auth")      // base path for every method in this class
public class AuthController {

    private final AuthService authService;
    public AuthController(AuthService authService) {   // constructor injection
        this.authService = authService;                // Spring passes the AuthService in
    }

    @PostMapping("/register")          // POST /api/v1/auth/register
    public ApiResponse<AuthResponse> register(@Valid @RequestBody RegisterRequest request) {
        return ApiResponse.ok(authService.register(request));
    }
}
```

Annotations to learn here:

- **@RestController** — this class handles HTTP and returns data (not HTML pages). Return values are auto-converted to JSON.
- **@RequestMapping("/api/v1/auth")** — prefix for all routes in the class.
- **@PostMapping("/register")** — handles HTTP POST to `/api/v1/auth/register`. (Siblings: `@GetMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`.)
- **@RequestBody** — "parse the JSON body into a `RegisterRequest` object."
- **@Valid** — "validate that object first" (triggers the `@NotBlank` / `@Email` rules). If it fails, the request never reaches your code — the global handler returns a 400.
- **Constructor injection** — the controller doesn't `new` an `AuthService`; Spring creates one and hands it in. This is *dependency injection*, the core idea of Spring.

Notice the controller is **thin**: it just delegates to the service and wraps the result in `ApiResponse.ok(...)`. No logic here.

Layer 2 — DTO (RegisterRequest.java)

A plain data carrier for the incoming JSON, with validation rules as annotations:

```
public class RegisterRequest {
    @NotBlank private String companyName;           // must not be empty
    @NotNull  private CompanyType companyType;       // must be present
    @NotBlank @Email private String email;           // must be a valid email
    @NotBlank @Size(min = 8) private String password; // ≥ 8 chars
    // getters / setters ...
}
```

Why DTOs exist: you never expose your database documents directly to the outside world. Incoming data → a Request DTO; outgoing data → a Response DTO. This keeps the API contract separate from your storage model (e.g. `passwordHash` never leaks out).

The matching output DTO uses a static **from(entity)** factory — a project convention:

```
public class UserResponse {
    public static UserResponse from(User user) {     // map document → safe DTO
        UserResponse r = new UserResponse();
        r.id = user.getId();
        r.email = user.getEmail();
        // note: passwordHash is deliberately NOT copied
        return r;
    }
}
```

Layer 3 — Service (AuthService.java)

The brain. Business rules + transactions live here.

```
@Service                              // a Spring-managed business component
public class AuthService {

    @Transactional                    // run inside a DB transaction (all-or-nothing)
    public AuthResponse register(RegisterRequest req) {
        if (userRepository.existsByEmail(req.getEmail()))      // rule: no duplicate emails
            throw new BusinessException("EMAIL_ALREADY_REGISTERED", "...");

        Company company = new Company();
        company.setName(req.getCompanyName());
        company = companyRepository.save(company);             // write to Mongo

        User user = new User();
        user.setEmail(req.getEmail());
        user.setPasswordHash(passwordEncoder.encode(req.getPassword())); // never store raw pw
        user.setRole(Role.COMPANY_ADMIN);
        user.setCompany(company);
        user = userRepository.save(user);                      // write to Mongo

        String token = jwtService.generateToken(               // issue a JWT
            user.getEmail(), user.getId().toString(), user.getRole().name());
        return new AuthResponse(token, UserResponse.from(user));
    }
}
```

- **@Service** — same as `@Component`, but signals "this is business logic." Spring auto-registers it for injection.
- **@Transactional** — wraps the method in a MongoDB transaction. If anything throws partway, both saves roll back (no half-created company). *Project gotcha:* most services must name their transaction manager, e.g. `@Transactional(transactionManager = "catalogTransactionManager")`. Identity is the exception — it has a single tx-manager bean so a plain `@Transactional` resolves fine.
- Notice it **registers a company AND user**, then **generates a JWT**. The JWT is the login token the client uses on every future request.

- **throw new BusinessException(...)** — you don't build error JSON by hand. You throw a typed exception; the global handler turns it into the right HTTP status + envelope.

Layer 4 — Repository (UserRepository.java)

The data layer — and you barely write any code. Spring Data generates the implementation from the method *names*.

```
public interface UserRepository extends MongoRepository<User, UUID> {
    Optional<User> findByEmail(String email);   // Spring writes the query from the name
    boolean existsByEmail(String email);

    @Query(value = "{ 'company.$id' : ?0 }", exists = true)   // custom query when needed
    boolean existsByCompanyId(UUID companyId);
}
```

- **MongoRepository<User, UUID>** — extend this interface and you instantly get `save()`, `findById()`, `findAll()`, `deleteById()`, etc., for a `User` document keyed by `UUID`. **You implement nothing.**
- **Derived queries**: `findByEmail` → Spring parses the name and builds the Mongo query automatically.
- **@Query** — drop to a raw Mongo query for things the name-parser can't express (here, traversing a `@DBRef`).

Layer 5 — Domain (User.java, Role.java)

The actual document stored in MongoDB.

```
@Document(collection = "users")        // → the "users" collection in Mongo
public class User extends BaseEntity {  // BaseEntity gives id, version, timestamps

    private String name;

    @Indexed(unique = true)            // build a unique index on email
    private String email;

    private String passwordHash;
    private Role role;                 // enum, stored as its String name
    private boolean active = true;

    @DBRef                             // a reference to another collection (companies)
    private Company company;
    // getters / setters ...
}
```

- **@Document(collection = "users")** — marks this as a Mongo document and names the collection.
- **extends BaseEntity** — shared base class (in `libs/common`) giving every document:

```
@Id  private UUID id = UUID.randomUUID();    // app-assigned UUID
@Version private Long version;              // optimistic locking
@CreatedDate  private Instant createdAt;    // auto-set on insert
@LastModifiedDate private Instant updatedAt; // auto-set on update
```

> **Subtle but important:** `version` is a **nullable Long**, not primitive `long`. Spring Data uses `null` = "new, insert me" vs non-null = "update me." A primitive `0` would be misread as new on every save.

- **@Indexed(unique = true)** — enforces no two users share an email (built on startup because `auto-index-creation: true`).
- **@DBRef** — `company` points to a document in the `companies` collection (a foreign-key-like link *within* one service). Links *across* services are never `@DBRef` — they're a bare `UUID` you resolve over REST.

> **The "no dirty checking" gotcha:** unlike JPA, Mongo won't auto-save a loaded-and-modified document. If a service method changes a `User`, it **must** call `userRepository.save(user)` explicitly or the change is lost.

config/IdentityMongoConfig.java

Per-service Mongo wiring:

```
@Configuration
@EnableMongoAuditing                  // makes @CreatedDate/@LastModifiedDate work
@EnableMongoRepositories(basePackages = "com.eximplatform.identity.repository")
public class IdentityMongoConfig {
    @Bean
    public MongoTransactionManager identityTransactionManager(MongoDatabaseFactory f) {
        return new MongoTransactionManager(f);   // the tx manager @Transactional uses
    }
}
```

- **@Configuration** — a class that defines beans.
- **@Bean** — the method's return value becomes a Spring-managed object (the transaction manager).
- **@EnableMongoRepositories** — tells Spring where the repository interfaces live so it can generate their implementations.

application.yml — the config

```
server:
  port: 8081                          # this service's port
spring:
  mongodb:
    uri: ${MONGODB_URI:mongodb://localhost:27017/...}   # the cluster (shared)
    database: ${IDENTITY_DB_NAME:exim_identity}         # but its OWN database
app:
  jwt:
    secret: ${JWT_SECRET:change-me...}                  # MUST match all services
  security:
    public-paths: /api/v1/auth/**     # endpoints that don't require a token
```

Every service shares the **same MONGODB_URI and JWT_SECRET** but uses its **own database** — that's the "one cluster, one database per service" design.

Part 3 — Shared Libraries

These live in `libs/` and are reused by all 14 services — they're how security and consistency happen everywhere at once.

libs/security — JWT auth (the same for every service)

A request arrives with header `Authorization: Bearer <token>`. Three pieces handle it.

JwtService — issues and verifies tokens (HS256)

```
public String generateToken(email, userId, role) {     // called at login/register
    return Jwts.builder().subject(email)
        .claim("uid", userId).claim("role", role)
        .expiration(...).signWith(key).compact();
}
public Claims parse(String token) { ... }               // verify signature, read claims
```

JwtAuthenticationFilter — runs on every request, before the controller

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    protected void doFilterInternal(req, res, chain) {
        String header = req.getHeader("Authorization");
        if (header startsWith "Bearer ") {
            Claims claims = jwtService.parse(token);          // validate token
            String role = claims.get("role", String.class);
            // tell Spring Security "this user is authenticated, with this role"
            SecurityContextHolder.getContext().setAuthentication(
                new UsernamePasswordAuthenticationToken(email, null,
                    List.of(new SimpleGrantedAuthority("ROLE_" + role))));
        }
        chain.doFilter(req, res);   // continue
    }
}
```

Key idea: **stateless**. There's no session and no DB lookup — the token itself carries who you are and your role. **Every service validates tokens independently** using the shared secret. That's why there's no central auth server on the request path.

SecurityConfig — the security rules, shared

```
@Configuration
@EnableMethodSecurity                  // enables @PreAuthorize on methods
public class SecurityConfig {
    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) {
        http.csrf(disable)
            .sessionManagement(STATELESS)         // no sessions
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(publicPaths).permitAll()   // /auth/**, swagger, health
                .anyRequest().authenticated())              // everything else needs a token
            .addFilterBefore(jwtAuthenticationFilter, ...); // plug in our JWT filter
        return http.build();
    }
}
```

Because `@EnableMethodSecurity` is on, services protect individual endpoints with:

```
@PreAuthorize("hasRole('PLATFORM_ADMIN')")   // only platform admins may call this
```

The roles are `COMPANY_ADMIN, COMPANY_MEMBER, PLATFORM_ADMIN, SUPPORT`, and the JWT stores them as `ROLE_<role>`.

libs/common — consistency everywhere

ApiResponse<T> — every endpoint returns the same envelope

```
{ "success": true, "data": {...}, "error": null, "timestamp": "..." }
```

Controllers call `ApiResponse.ok(data)`; errors use `ApiResponse.fail(error)`.

GlobalExceptionHandler — one class catches exceptions from all controllers

```
@RestControllerAdvice                  // applies to every controller in the app
public class GlobalExceptionHandler {
    @ExceptionHandler(BusinessException.class)       // 422
    @ExceptionHandler(NotFoundException.class)       // 404
    @ExceptionHandler(AccessDeniedException.class)   // 403 (a @PreAuthorize denial)
    @ExceptionHandler(MethodArgumentNotValidException.class) // 400 (@Valid failed)
    // ... each returns ApiResponse.fail(...) with the correct status
}
```

This is why your services just `throw new BusinessException(...)` and never build error JSON by hand — **@RestControllerAdvice** intercepts it and formats the response consistently.

How a Request Flows End-to-End (register)

```
POST /api/v1/auth/register  { companyName, email, password, ... }
   │
   ▼  GATEWAY :8080
      matches Path=/api/v1/auth/**  → forwards to identity :8081 (keeps Authorization header)
   │
   ▼  identity-service
      JwtAuthenticationFilter   → /auth/** is public, so no token needed; passes through
      AuthController.register   → @Valid checks the DTO, delegates to the service
      AuthService.register      → @Transactional: checks duplicate email,
                                  saves Company + User, hashes password, issues JWT
      Repository.save           → writes to MongoDB (exim_identity database)
      UserResponse.from(user)   → maps document → safe DTO (no passwordHash)
   │
   ▼  returns ApiResponse.ok({ accessToken, user })
```

On the next request the client sends `Authorization: Bearer <accessToken>`, the JWT filter validates it, and `@PreAuthorize` enforces the user's role.

The Mental Model to Keep

Annotation / piece	What it does
`@SpringBootApplication`	App entry point; turns on auto-config + component scanning
`@RestController` + `@RequestMapping` / `@PostMapping`	Maps URLs → methods, returns JSON (web layer)
`@RequestBody` / `@Valid`	Parse + validate incoming JSON into a DTO
`@Service`	Business logic class
`@Transactional`	Run the method in a DB transaction (all-or-nothing)
`MongoRepository<T, ID>`	Free CRUD + derived queries (data layer)
`@Document` / `@Indexed` / `@DBRef`	Mongo document, index, intra-service reference
`extends BaseEntity`	Shared id/version/timestamps
DTO + `from(entity)`	Decouple API shape from storage; hide secrets
`@Configuration` / `@Bean`	Define Spring-managed objects
`@RestControllerAdvice`	One place to turn exceptions into HTTP responses
JWT filter + `@PreAuthorize`	Stateless auth; per-endpoint role checks
Constructor injection	Spring supplies dependencies; you never `new` them

Every one of the 14 services follows this exact same skeleton — once you understand identity-service, you understand all of them; only the domain (orders, catalog, payments…) changes.